

OBJECTIVE

ML Engineer with 1+ year of experience in RAG and multi-agent systems, focused on building scalable and reliable AI solutions for production-oriented workflows.

EDUCATION

Belarusian State University of Informatics and Radioelectronics (BSUIR)

July 2022 – June 2026

GPA: 8.0/10 — 4th year student

TECHNICAL EXPERTISE

Core Competencies	LLM Engineering: RAG architectures, Multi-agent systems, Prompt optimization, Semantic search, Embeddings Backend Development: REST API design, Microservices, Async processing, Service orchestration
Languages & Frameworks	Python (FastAPI, LangChain, LangGraph) LLM APIs: OpenAI, LLaMA, Prompt engineering patterns
Infrastructure & Tools	Databases: PostgreSQL, MongoDB, Chroma, FAISS DevOps: Docker, Git/GitHub, n8n

EXPERIENCE

Machine Learning Engineer

Nov 2024 – Dec 2025

IFortex

- Designed and deployed RAG systems with hybrid search and re-ranking to improve retrieval quality.
- Built multi-agent LangGraph workflows for automated CV screening in HR-tech pipeline.
- Developed FastAPI services for LLM integration with focus on low latency and fault tolerance.
- Integrated Llama/OpenAI models into products: prompt versioning and response validation.

Machine Learning Intern

Sep 2024 – Nov 2024

IFortex

- Developed internal chatbots using OpenAI API for customer support automation.
- Implemented conversational pipelines with prompt engineering and basic retrieval.
- Tested model response quality and refined prompts based on results.

PROJECTS

Multi-Agent CV Screening

Domain – HR Tech / Recruitment Automation

A multi-agent platform designed to automate large-scale CV screening, structure unstructured candidate data, and match profiles to job requirements with high accuracy. The system combines agent workflows, semantic retrieval, and criteria-based ranking to streamline decision-making for recruiters. Architecture includes orchestrated pipelines for ingestion, parsing, and ranking, supported by vector search and real-time evaluation endpoints.

Responsibilities (Process & Architecture)

- Implemented a multi-agent workflow in LangGraph for automated CV parsing and ranking.
- Designed ingestion pipelines for structured and unstructured text, including parsing, normalization, and embedding-based processing.
- Optimized query relevance and latency through vector database tuning and retrieval ranking.
- Implemented caching mechanisms for repeated or similar requests to reduce token usage and improve responsiveness.
- Integrated PII detection and redaction to ensure compliance before storing or processing candidate data.

Technologies: Python, FastAPI, LangChain, LangGraph, PostgreSQL, Chroma, Docker, spaCy/Presidio (PII detection), OpenAI API

AI Summarization Service

Domain – Document Processing / Legal Tech

A modular summarization platform designed to process large volumes of legal and business documents, extracting key insights and generating concise summaries. The system supports interchangeable LLM backends (cloud and local), with configurable summary modes (abstractive, extractive, bullet-point). Built with production-grade optimizations including caching, batching, and rate limiting to minimize API costs and latency.

Responsibilities (Process & Architecture)

- Built modular summarization pipeline with pluggable LLM backends (OpenAI, T5, Llama).
- Integrated Gradio for rapid prototyping and A/B testing of different summary modes.
- Implemented caching, request batching, and token-based rate limiting to reduce API overhead by optimizing repeated calls.
- Created versioned prompt templates and established performance benchmarks for quality evaluation.
- Containerized the service with Docker for consistent deployment across environments.
- Designed REST API endpoints with FastAPI for seamless integration with downstream services.

Technologies: Python, FastAPI, Gradio, OpenAI API, HuggingFace T5, Docker, Redis (caching), Prometheus (monitoring)

RAG for Medical Documentation

Domain – Healthcare / Clinical Decision Support

A domain-specific RAG system designed to assist medical professionals in retrieving relevant information from clinical guidelines, research papers, and patient records. The system implements advanced chunking strategies optimized for medical terminology, hybrid search combining dense and sparse retrieval, and source attribution for compliance with healthcare standards. Designed with focus on explainability and low-latency responses for point-of-care use.

Responsibilities (Process & Architecture)

- Developed end-to-end RAG pipeline with medical domain adaptation: custom chunking strategies for clinical texts and terminology-aware preprocessing.
- Implemented hybrid search combining dense embeddings (FAISS) with keyword-based retrieval (BM25) for improved recall on medical queries.
- Designed prompt templates with chain-of-thought reasoning and source citation for explainable AI responses.
- Optimized retrieval latency through vector indexing and embedding caching.
- Built evaluation framework for assessing answer accuracy and relevance using domain expert feedback.
- Containerized the solution with Docker and implemented basic monitoring for production deployment.

Technologies: Python, LangChain, FAISS, PostgreSQL, Docker, OpenAI API, Llama, HuggingFace Embeddings, Redis (caching)